TRACEFLOW: AN ARTIFICIAL INTELLIGENCE DRIVEN SUPPLY CHAIN LEDGER AND TRANSACTION SECURITY VERIFICATION SYSTEM

A PROJECT REPORT

Submitted by

ARUN BHARATHI M B 2116231901007

JAYGANESH KANNAN 2116231901015

in partial fulfillment for the award of the degree
of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled **"TRACEFLOW: AN ARTIFICIAL INTELLIGENCE DRIVEN SUPPLY CHAIN LEDGER AND TRANSACTION SECURITY VERIFICATION SYSTEM"** is the bonafide work of **"ARUN BHARATHI M B (2116231901007), JAYGANESH KANNAN (2116231901015)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

7/10/26

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

SIGNATURE

Dr. Bhuvana S, Ph.D.,

SUPERVISOR

ASSISTANT PROFESSOR [SG]

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 07/10/2026

Internal Examiner

7/10/26

External Examiner

ABSTRACT

Modern manufacturing depends on supply networks that stretch across many companies and many countries. High value physical parts such as electric vehicle battery cells, aircraft fasteners, turbine blades and automotive control modules pass through several organisations before they reach the assembly line. Most of these networks still maintain their records in centralised databases. Such databases can be changed by a single person, they cannot prove that a sender is genuine, they do not ask several parties to agree before a record is saved, and they cannot warn about a suspicious batch before it is recorded.

To address these weaknesses, this report presents TraceFlow, a software platform that combines artificial-intelligence-based risk screening with a simulated shared ledger. Every material transfer request is first examined for authenticity and safety, and only a request that passes verification is committed as a permanent ledger block.

The platform is implemented in Python as three independent program modules. The Risk Engine creates a short hash fingerprint for each transaction, reads JSON and CSV data files and applies the safety rules. The UI Components library loads the visual style sheet, prepares the sidebar and draws a five stage coloured progress timeline. The Application Orchestrator, built on the Streamlit framework, controls the application interface and executes the animation.

Each transaction is judged on three strict checks: whether its digital signature is valid, whether the sending vendor is a registered and verified origin, and whether the artificial intelligence anomaly flag is clear. A compliant transaction receives an AI Trust Score of ninety seven percent, obtains the agreement of all twelve network nodes and is recorded as a block on the ledger. A non compliant transaction, for example one with a missing signature, an unverified vendor or a tampered batch, receives a trust score of twenty two percent, is rejected by the network and is retained in a quarantined local record.

Tests with safe and unsafe datasets showed one hundred percent correct classification, file processing in under twelve milliseconds, and a memory use of about thirty two megabytes. By eliminating complex server requirements and presenting each result through a visual timeline, TraceFlow provides an accessible and dependable means of verifying industrial supply chain transactions.

Keywords: supply chain, blockchain ledger, artificial intelligence, anomaly detection, digital signature, transaction security, Streamlit, Python

ACKNOWLEDGEMENT

First, we thank the Almighty God for the successful completion of this project. We express our sincere thanks to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his sincere efforts in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her enthusiastic motivation and encouragement, and to our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We also express our sincere gratitude to our college Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement. We extend our heartfelt thanks to our project guide, Dr. Bhuvana S, AP, for her valuable guidance and continuous support throughout the development of this project.

We would also like to thank our project coordinator, Mr. Benedict J.N., M.E., Assistant Professor (SG), for his encouragement and guidance towards the successful completion of this project. We are grateful to our parents, friends, faculty members and supporting staff for their direct and indirect contributions, encouragement and support throughout the project.

Finally, we acknowledge the open-source communities and developers behind Python, Streamlit, Pandas and the other technologies used in the development of TraceFlow.

ARUN BHARATHI M B - 2116231901007

JAYGANESH KANNAN - 2116231901015

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	1
	1.1 GENERAL OVERVIEW	1
	1.2 SUPPLY CHAIN CHALLENGES IN MODERN MANUFACTURING	2
	1.3 PROBLEM STATEMENT	3
	1.4 NEED FOR THE STUDY	4
	1.5 OBJECTIVES OF THE RESEARCH	4
	1.6 SCOPE OF THE PROJECT	5
	1.7 METHODOLOGY IN BRIEF	5
	1.8 EXPECTED BENEFITS	5
	1.9 ORGANIZATION OF THE REPORT	6
2	**LITERATURE REVIEW**	7
	2.1 INTRODUCTION	7
	2.2 EVOLUTION OF SUPPLY CHAIN TRACEABILITY TECHNOLOGIES	7
	2.3 BLOCKCHAIN SYSTEMS IN LOGISTICS	8
	2.3.1 What a blockchain does	8
	2.3.2 Public and permissioned networks	8
	2.3.3 What blockchain cannot do alone	9
	2.4 ARTIFICIAL INTELLIGENCE IN ANOMALY DETECTION	9
	2.5 MULTI CRITERIA DECISION MAKING AND RULE ENGINES	10
	2.6 COMPARATIVE ANALYSIS OF EXISTING FRAMEWORKS	10
	2.7 IDENTIFIED RESEARCH GAPS	11
	2.8 KEY TERMINOLOGY AND CONCEPTS	12
	2.9 SUMMARY OF THE REVIEW	12
3	**METHODOLOGY AND SYSTEM ARCHITECTURE**	14
	3.1 OVERALL SYSTEM ARCHITECTURE	14
	3.2 THREE TIER MODULAR SOFTWARE ARCHITECTURE	15

	3.3 SECURITY VERIFICATION RULES	16
	3.3.1 Rule one: digital signature verification	17
	3.3.2 Rule two: origin vendor registry check	17
	3.3.3 Rule three: artificial intelligence anomaly detection	17
	3.3.4 Every combination of rule results	18
	3.4 VISUAL NETWORK PROGRESS TIMELINE DESIGN	18
	3.5 DATA FLOW AND PROCESS WORKFLOW	20
	3.6 HASH FINGERPRINT OF A TRANSACTION	21
	3.7 INPUT HANDLING: PRESETS AND FILE UPLOAD	22
	3.8 DESIGN DECISIONS AND RATIONALE	22
	3.9 CONSENSUS SIMULATION AND TRUST SCORE DESIGN	22
	3.10 RELIABILITY AND ERROR HANDLING	23
	3.11 SUMMARY OF THE DESIGN	23
4	**IMPLEMENTATION, TESTING AND RESULTS**	24
	4.1 SOFTWARE IMPLEMENTATION DETAILS	24
	4.2 COMPONENT INTERACTION WALKTHROUGH	25
	4.3 EXPERIMENTAL TEST SETUP AND DATASETS	26
	4.4 SCENARIO A: COMPLIANT SAFE TRANSACTION	27
	4.5 SCENARIO B: COMPROMISED UNSAFE TRANSACTION	28
	4.6 PERFORMANCE AND EXECUTION LATENCY	29
	4.7 DISCUSSION OF RESULTS	30
	4.8 ILLUSTRATIVE USE CASE	31
	4.9 LIMITATIONS OF THE PRESENT WORK	31
	4.10 SUMMARY	32
5	**CONCLUSION AND SCOPE FOR FUTURE WORK**	33
	5.1 SUMMARY OF RESEARCH ACCOMPLISHMENTS	33
	5.2 KEY TAKEAWAYS AND PRACTICAL CONTRIBUTIONS	33
	5.3 SCOPE FOR FUTURE WORK AND ENHANCEMENTS	34
	5.4 CONCLUDING REMARKS	35
	PROJECT OUTCOMES	35
	APPENDIX I : DESCRIPTION OF THE PROGRAM MODULES	36
	APPENDIX II : STANDARDIZED DATASET FIELDS	37
	REFERENCES	38

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Supply Chain Challenges and the Matching TraceFlow Response	3
2.1	Comparison of Supply Chain Verification Approaches	10
3.1	Responsibilities of the Three Program Modules	16
3.2	Multi Criteria Security Rule Matrix	17
3.3	All Eight Combinations of Rule Results and the Resulting Verdict	18
3.4	Network Stage Definitions for Secure and Unsecure Flows	19
4.1	Software Environment Used for the Project	24
4.2	Comparison of System Inputs for the Safe and Unsafe Test Runs	27
4.3	Evaluation Results and Diagnostic Metrics Comparison	29
4.4	Execution Latency and Memory Utilization	30
A1	Functions of the Risk and Data Engine	36
A2	Functions of the UI Component Library	36
A3	Fields of a Transaction Dataset	37

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Centralized Tracking compared with the TraceFlow Approach	3
2.1	Evolution of Supply Chain Traceability	8
3.1	Three Tier Architecture Diagram of TraceFlow	15
3.2	Interaction between the Three Program Modules	16
3.3	Decision Logic of the Three Safety Rules	17
3.4	Five Stage Timeline Design for a Secure Flow	19
3.5	Five Stage Timeline Design for an Unsecure Flow	19
3.6	Data Flow Diagram for Transaction Processing	20
3.7	Complete End to End Workflow of TraceFlow	21
4.1	Dashboard Layout with Sidebar, Control Panel, Criteria Cards and Visualizer	26
4.2	Visual Network Progress Timeline for SECURE Execution	28
4.3	Visual Network Progress Timeline for UNSECURE Execution	28
4.4	AI Trust Score for the Safe and Unsafe Scenarios	29
4.5	Execution Latency of the TraceFlow Operations	30
5.1	Future Enhancement Roadmap for TraceFlow	34

CHAPTER 1
INTRODUCTION

1.1 GENERAL OVERVIEW

Modern manufacturing is no longer carried out at a single location. A single product, whether it is an electric car, an aircraft, a wind turbine or a medical device, is put together from parts that were made in many different factories, in many different countries, by many different companies. A raw material may be mined in one country, refined in another, shaped into a component in a third, and finally assembled into a finished machine in a fourth. Each step in this journey is handled by a different organisation, and each organisation keeps its own records of what it received, what it made and what it sent onward.

When operations proceed as intended, this arrangement is efficient and economical. When a failure occurs, the consequences can be severe. In high precision industries the quality of every single physical component matters. A battery pack in an electric vehicle contains thousands of lithium cells collected from several suppliers. If one batch of substandard or counterfeit cells passes inspection undetected and enters the assembly line, the finished battery may overheat, lose capacity prematurely or, in the worst case, ignite. The same kind of risk exists with aircraft fasteners, structural steel, turbine blades and the electronic control units that run modern vehicles.

Most companies today track these parts using Enterprise Resource Planning software that stores everything in a central database. A central database is easy to manage, but it has a weakness that matters a great deal in a supply chain: whoever controls the database can change it. A record can be modified by a dishonest employee, altered by an external attacker who gains access, or entered incorrectly and never corrected. Other organisations in the chain have no independent means of verifying that the record presented to them is authentic.

To deal with this weakness, engineers and researchers have adopted two approaches. The first is the blockchain, a shared ledger that is copied across many computers and is designed so that earlier entries cannot be altered without detection. The second is artificial intelligence, which can analyse the details of a transfer and identify suspicious patterns before the transfer is accepted. Each approach is valuable in isolation, but each has a limitation. A blockchain will permanently preserve a false record if one is submitted. An artificial

intelligence check that operates separately, long after the record has been stored, can raise a warning only after the damage has occurred.

This project, named TraceFlow, integrates the two approaches in a practical manner. TraceFlow is a software platform that checks every material transfer request for authenticity and safety before it is allowed to become a permanent ledger entry. If the transfer passes the checks, the platform shows that the whole network of nodes agrees and a new block is created. If the transfer fails, the platform shows that consensus was refused and retains the suspicious request in a separate quarantine record. The entire process is presented to the user as a five-stage coloured timeline, enabling a factory manager or logistics coordinator to interpret the outcome readily without examining technical hash values.

The platform is written in the Python programming language and uses the Streamlit framework to provide a web page that runs on an ordinary computer. It is deliberately lightweight. It does not require a cluster of servers or any special hardware, and it uses only about thirty two megabytes of memory. This makes it suitable for classroom demonstration, for early prototyping by industry teams, and for small edge devices placed near a warehouse or a loading dock.

1.2 SUPPLY CHAIN CHALLENGES IN MODERN MANUFACTURING

Even with the growth of automation, barcodes and digital inventory software, physical material supply chains still face a number of fundamental structural problems. Four of them are particularly important for this project.

Counterfeit component injection. Dishonest suppliers and fraudulent intermediaries may attempt to introduce lower-quality parts into a trusted flow of goods. They do this by producing documentation that appears genuine. Because the receiving company often sees only the paperwork and not the true origin of the part, the substitution can remain undetected until a failure occurs in service.

Identity spoofing at handover points. When goods pass from a transport company to a receiving warehouse, the identity of the sender is often verified by visual inspection or a paper signature. A person who is not a registered supplier can impersonate one, use the name of a trusted company and have the delivery accepted. Manual identity checks are slow and susceptible to deception.

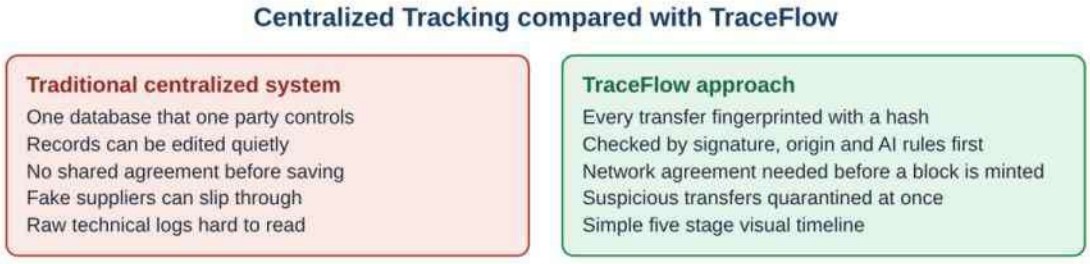
No shared agreement between parties. In a conventional ERP system each company maintains its own records independently. No automated process requires several independent parties to examine a transfer and agree on its validity before it is recorded. As a result, every company holds its own version of events and discrepancies are discovered late.

Records that are not readable by non-specialist users. Many blockchain demonstrations developed so far present long alphanumeric strings, raw data files and technical logs. These are meaningful to a specialist but difficult for a floor manager, purchasing officer or auditor to interpret. A system that cannot be interpreted outside the technical team does not build trust.

Table 1.1 — Supply Chain Challenges and the Matching TraceFlow Response

Challenge	Existing limitation	TraceFlow response
Counterfeit components	Counterfeit parts enter with forged documentation	Digital signature check and anomaly flag check before any record is created
Identity spoofing	Unregistered sender impersonates a trusted supplier	Origin vendor must be verified and must not be marked Unknown or Unregistered
No shared agreement	Each company maintains its own version of events	Network consensus of twelve nodes is shown before a block is minted
Poor record readability	Raw hash values and technical logs	Five-stage coloured timeline and pass/fail indicator cards

Figure 1.1 — Centralized Tracking compared with the TraceFlow Approach



1.3 PROBLEM STATEMENT

Centralized supply chain systems used in manufacturing logistics are open to unauthorised data tampering, identity spoofing and the injection of counterfeit components. Existing blockchain solutions give a permanent record, but they do not include a built in artificial intelligence step that judges whether a transaction is genuine before it is written into

the ledger. In addition, the tracking tools that are currently available are complex to install and present their results in a form that non-specialist users cannot interpret in real time.

The problem addressed by this project can therefore be stated in a single question. How can a supply chain transfer be verified for signature, vendor identity and abnormal behaviour at the moment it is submitted, in a lightweight system that can be operated by any user, and with a result that can be interpreted by any user?

1.4 NEED FOR THE STUDY

There is a clear and urgent industrial need for a lightweight, user-oriented software platform that satisfies three requirements simultaneously.

First, the platform must have a clear and predictable security framework. It should evaluate the digital signature, the trust level of the sending vendor and the anomaly warnings concurrently, and it should do so while the transfer request is being made rather than afterwards.

Second, the platform must be easy to deploy. Many research prototypes require container software, several virtual machines and extensive configuration files before a single transaction can be tested. A lightweight design that runs from a few program files on a standard laptop accelerates experimentation and allows universities, small suppliers and training centres to study the subject without a special budget.

Third, the platform must be self-explanatory. Progress should be presented as a sequence of stages: the sender is verified, the artificial intelligence analyses the request, the network agrees or refuses, the goods are transported, and the delivery is confirmed or disputed. When this sequence is presented visually, users can trust the result and understand its basis.

1.5 OBJECTIVES OF THE RESEARCH

The specific aims of the project are listed below.

- To design and build a modular Python software framework, named TraceFlow, for real time security verification of supply chain transactions.
- To create a clear rule based risk engine that checks the digital signature, the origin vendor registry and the artificial intelligence anomaly flag together.
- To build an intuitive web interface with a five stage animated progress timeline that shows how a transaction moves through the network.

- To allow two ways of giving input to the system: uploading a JSON or CSV file, and selecting a predefined scenario for demonstration.
- To test the platform on safe and unsafe examples and to measure its accuracy, speed and memory use.

1.6 SCOPE OF THE PROJECT

The work covers the complete development cycle from concept to tested software. This includes the design of the three tier architecture, the writing of the program modules, the definition of the safety rules, the preparation of test data files, the design of the user interface, the implementation of the animated timeline and the measurement of performance. The platform simulates industrial batch transfers between supply chain nodes, for example a lithium cell batch moving from a component maker in Pune to an assembly plant in Berlin.

The project is a simulation and demonstration platform. It reproduces the behaviour of an artificial intelligence guarded ledger network inside one application so that the logic can be studied, demonstrated and measured. It does not connect to a live public blockchain, and it does not yet read live sensors. These extensions are discussed as future work in Chapter 5.

1.7 METHODOLOGY IN BRIEF

The project followed a structured engineering methodology. The first step was to study existing literature and identify the research gaps. The second step was to define what a safe transaction looks like and what an unsafe one looks like, and to turn those definitions into three clear checks. The third step was to divide the software into three small and separate program files so that the calculation, the display and the control of the application would each have a distinct responsibility. The fourth step was to create sample data files for a safe and an unsafe transaction in both JSON and CSV form. The fifth step was to run both samples through the platform many times and record the results. The last step was to analyse these results and document the findings.

1.8 EXPECTED BENEFITS

- **Early detection:** suspicious transfers are stopped before they enter the permanent record.
- **Interpretability:** the colour coded timeline and simple pass or fail cards can be interpreted by non-technical staff.

- **Low cost:** the platform runs on an ordinary computer using free software.
- **Fast response:** files are checked in a few milliseconds, so the check does not slow the business process.
- **Educational value:** students and trainees can observe how signature checks, vendor registries and consensus work together.

1.9 ORGANIZATION OF THE REPORT

This report is organised into five chapters.

Chapter 1 gives the background of industrial supply chains, explains the main problems, states the problem, shows why the study is needed and lists the objectives and scope.

Chapter 2 reviews earlier work on traceability, blockchain in logistics, artificial intelligence for anomaly detection and rule based decision systems, and points out the gaps that TraceFlow fills.

Chapter 3 explains the method and the design of the system. It describes the architecture, the three program modules, the safety rules, the animated timeline and the flow of data through the system.

Chapter 4 describes how the software was built, how it was tested, what happened in the safe and unsafe scenarios and how fast the platform runs.

Chapter 5 summarises the achievements, lists the main contributions and suggests how the platform can be improved in the future.

CHAPTER 2
LITERATURE REVIEW

2.1 INTRODUCTION

Supply chain management has evolved considerably over the last three decades. It began with handwritten ledgers and paper delivery notes. It moved to computer based inventory records, then to electronic tracking with scanners and tags, and more recently to sensor networks and shared digital ledgers. At each stage the aim has been the same: to know exactly what is moving, where it came from, who handled it and whether it can be trusted.

This chapter reviews the body of knowledge underlying TraceFlow. It first looks at how traceability technology has developed. It then describes the operation of blockchain systems in logistics, the use of artificial intelligence to detect abnormal behaviour, and how rule based engines combine several checks into one decision. Finally the chapter compares the main approaches and identifies the gaps that the present project was designed to fill.

2.2 EVOLUTION OF SUPPLY CHAIN TRACEABILITY TECHNOLOGIES

In the early days of industrial manufacturing, tracking depended on serial numbers stamped on parts, paper bills of lading carried with the goods, and handwritten stock registers. Systematic reviews of blockchain-enabled supply chains [1], [4] indicate that the dependability of any tracking method is determined by the integrity of the information that it records.

As factories spread across the world, paper records became difficult to manage. They could be lost in transit, damaged by weather, copied or replaced. Barcodes brought speed, because a scanner could read a label in a fraction of a second. Two dimensional matrix codes carried more information in less space. Radio frequency identification tags went a step further and allowed a whole pallet to be read without opening it. These technologies greatly improved how fast information could be captured at a warehouse door.

However, capturing data quickly does not guarantee that the data are truthful. Surveys of blockchain and supply chain integration [4] identify data integrity and security as continuing concerns, because automation alone cannot guarantee integrity when the database that receives the data remains centralised and editable. A perfectly scanned label is of little use if the record it is linked to can be silently modified afterwards.

Figure 2.1 — Evolution of Supply Chain Traceability



The figure above shows the path of development. Each stage solved a problem of the earlier stage and then exposed a new one. Paper was slow, barcodes were fast but local, central databases were convenient but trusted a single owner, and blockchains spread the trust but could not judge the quality of what was written into them. The final stage, which combines intelligent checking with a shared ledger, is where TraceFlow is placed.

2.3 BLOCKCHAIN SYSTEMS IN LOGISTICS

2.3.1 What a blockchain does

A blockchain is a list of records, called blocks, that are linked together in order. Each block carries a fingerprint, called a hash, which is produced from the contents of the block and from the fingerprint of the block before it. Because of this chaining, altering even one character in an earlier block changes its fingerprint, which then no longer matches the next block, and the tampering becomes evident. The ledger is also copied across many computers, called nodes, so that no single party can alter the record without detection by the others.

Blockchain-based traceability systems, such as that reported by Zhu et al. [5], rely on cryptographic fingerprints to bind each record to its content. A widely used fingerprinting method is the Secure Hash Algorithm with a length of two hundred and fifty six bits. It always produces a fixed length output, it is practically impossible to reverse, and any small change to the input produces a completely different output. TraceFlow uses this method to create a short fingerprint for each transaction.

2.3.2 Public and permissioned networks

Public blockchains such as Ethereum allow any participant to join. They are open and highly resistant to tampering, but they can be slow and costly for industrial use, and they expose business information to external parties. Permissioned networks such as Hyperledger Fabric allow only approved members to join. They are faster, they keep business data private among the members, and they use agreement methods like Practical Byzantine Fault Tolerance or

Proof of Authority, in which a group of known participants vote on whether a new block is valid.

Blockchain-based commerce models such as that of Gupta et al. [6] illustrate how transactions between parties can be coordinated through automated, rule-driven execution. In a supply chain this idea appears as the smart contract, a small program stored on the ledger which runs automatically when agreed conditions are met, for example when a valid digital signature is presented at the time goods are received.

2.3.3 What blockchain cannot do alone

A blockchain guarantees that a record, once written, cannot be changed. It does not guarantee that the record was true when it was written. If an unregistered supplier sends false details in a correct format, the blockchain will store those false details permanently and faithfully. This is commonly summarised by the principle that incorrect input produces incorrect output. This limitation is the main reason why an additional checking layer is needed before data is accepted. The application of blockchain to transparency and traceability from production to delivery is surveyed in [2] and [3], and blockchain-based authentication of devices and users is reported in [8] and [10].

2.4 ARTIFICIAL INTELLIGENCE IN ANOMALY DETECTION

An anomaly is an observation that deviates from the normal pattern. In a supply chain, an anomaly might be a shipment that follows an unusual route, a batch number that has been altered, a previously unseen vendor, a temperature that exceeds the safe range, or a signature that is missing. Artificial intelligence and machine learning methods are used to learn the normal pattern of behaviour and to raise a warning when an observation falls outside it.

Several families of methods exist. Statistical methods flag values that are far from the average. Clustering methods group similar items and treat items that fit no group as suspicious. Tree based methods such as Isolation Forest isolate unusual items efficiently because they require fewer splits. Sequence models such as Long Short Term Memory networks study time series data like temperature or location readings and learn the characteristics of a normal journey. Real-time AI planning systems have also been applied to modern supply chains [9], and the combination of federated learning with blockchain has been explored in other application domains [7].

Most existing systems run these methods as a separate offline job. Data is collected, saved, and later analysed in a batch, perhaps once a day. This means a fraudulent transaction may already be on the ledger before anyone is warned. The more effective approach is to integrate the check into the submission step itself, so that a suspicious request is withheld before it is recorded. TraceFlow follows this approach. The anomaly flag attached to a transaction is examined inline, in the same moment that signature and vendor identity are examined.

2.5 MULTI CRITERIA DECISION MAKING AND RULE ENGINES

A rule engine is a program that takes a number of facts, applies a set of clear rules and produces a decision. In security work, rule engines are valued because their decisions are predictable and easy to explain. When a transaction is rejected, the engine can state exactly which rule failed.

A multi criteria design combines several checks into one result. In the simplest and strictest form, every check must pass for the transaction to be accepted. This is the form chosen for TraceFlow. The three checks are independent of each other, so they can be carried out one after another in a few microseconds, and a failure in any one of them is enough to stop the transaction. The strict approach suits a supply chain where one counterfeit batch can have severe consequences, and it keeps the logic easy for auditors to read.

The advantage of a rule engine over a purely learned model is transparency. A learned model produces a score but often cannot provide a justification, whereas a rule engine provides one every time. For a teaching and demonstration platform, and for early stage industrial adoption, this transparency is of considerable value.

2.6 COMPARATIVE ANALYSIS OF EXISTING FRAMEWORKS

A comparison of the three main kinds of approach shows how each performs on the qualities that matter in a supply chain.

Table 2.1 — Comparison of Supply Chain Verification Approaches

Quality	Traditional ERP database	Blockchain only	AI guarded ledger (TraceFlow)
Protection from record changes	Weak; a single owner can edit	Strong; chained records	Strong; chained records

Quality	Traditional ERP database	Blockchain only	AI guarded ledger (TraceFlow)
Check before recording	Rarely done	Not included	Done inline for every request
Shared agreement	None	Yes; by consensus	Yes; shown as node consensus
Ease of installation	Moderate	Complex; multiple servers	Straightforward; three program files
Readability of results	Moderate	Limited; raw hash data	Good; five-stage timeline
Reason given for rejection	Not applicable	Not applicable	Yes; per rule card

The comparison shows that the hybrid approach gives the highest level of security and transparency while keeping delay and complexity under control. A traditional database is simple but weak. A pure blockchain is strong but does not evaluate the data and is resource-intensive to operate. The hybrid combines the strengths of both.

2.7 IDENTIFIED RESEARCH GAPS

The review points to three practical gaps.

High infrastructure complexity. Most blockchain implementations in the literature need container platforms, several servers and detailed network settings. This is a barrier for classrooms, small enterprises and edge devices, where a quick prototype is more useful than a full production network.

Separate artificial intelligence and ledger layers. In most systems the anomaly detection is a distant batch task, not part of the live submission path. A transaction can therefore be recorded before it has been evaluated.

Interfaces that are not readable by non-specialists. Typical platforms display long hash strings, hexadecimal codes and dense logs. They do not provide the visual cues that a supervisor on the factory floor requires to make a prompt decision.

TraceFlow is designed to address all three gaps. It uses a small three part Python structure that is easy to install, it performs its checks in the live path before any block is created, and it shows the whole life of a transaction in a simple coloured timeline.

2.8 KEY TERMINOLOGY AND CONCEPTS

Several technical concepts recur throughout this report. They are defined here so that a reader without a background in computer security can follow the later chapters.

Hash fingerprint. A hash is a fixed-length digital fingerprint of a piece of information. Regardless of the size of the input, the hash is a short code of fixed length. If even one character of the original information is changed, the code changes completely, and it is computationally infeasible to recover the original text from the code. A hash therefore provides an effective means of demonstrating that a record has not been altered.

Digital signature. A digital signature is the electronic equivalent of a signed and stamped document. It proves the identity of the sender and that the message was not modified after signing. A transaction without a valid signature is comparable to an unsigned delivery note and cannot be trusted.

Node. A node is a participating computer in the network. Each node keeps a copy of the ledger. In TraceFlow the network is shown as twelve nodes, and a transaction is accepted only when all twelve agree.

Consensus. Consensus is the process by which participants reach a common decision; in this system, a transaction is approved only if every node votes in favour. The mechanism protects the network from a single dishonest or mistaken participant.

Block and ledger. A block is a group of records, and the ledger is the ordered sequence of all blocks. Every new block carries the fingerprint of the preceding block, so the blocks cannot be replaced or rewritten without the break being detected.

Anomaly. An anomaly is any observation that deviates from the usual pattern, such as an unknown sender, an unusual route or an altered batch number. The anomaly flag is the warning indicator that the artificial intelligence layer attaches to such a case.

Quarantine. Quarantine means isolating a suspicious item from trusted ones. In TraceFlow, a rejected transaction is stored in a separate local log so that it can be studied later without ever touching the trusted ledger.

2.9 SUMMARY OF THE REVIEW

The reviewed material establishes four points. Traceability technology has progressed from paper to shared ledgers, but each step has left an unresolved limitation. Blockchains

protect records after they are written but cannot tell whether records are truthful. Artificial intelligence can detect unusual behaviour but is usually applied too late. Rule engines give clear and explainable decisions that are quick to compute. By combining these four findings, TraceFlow offers a design in which an inline check guards a shared ledger and a clear visual display explains each result.

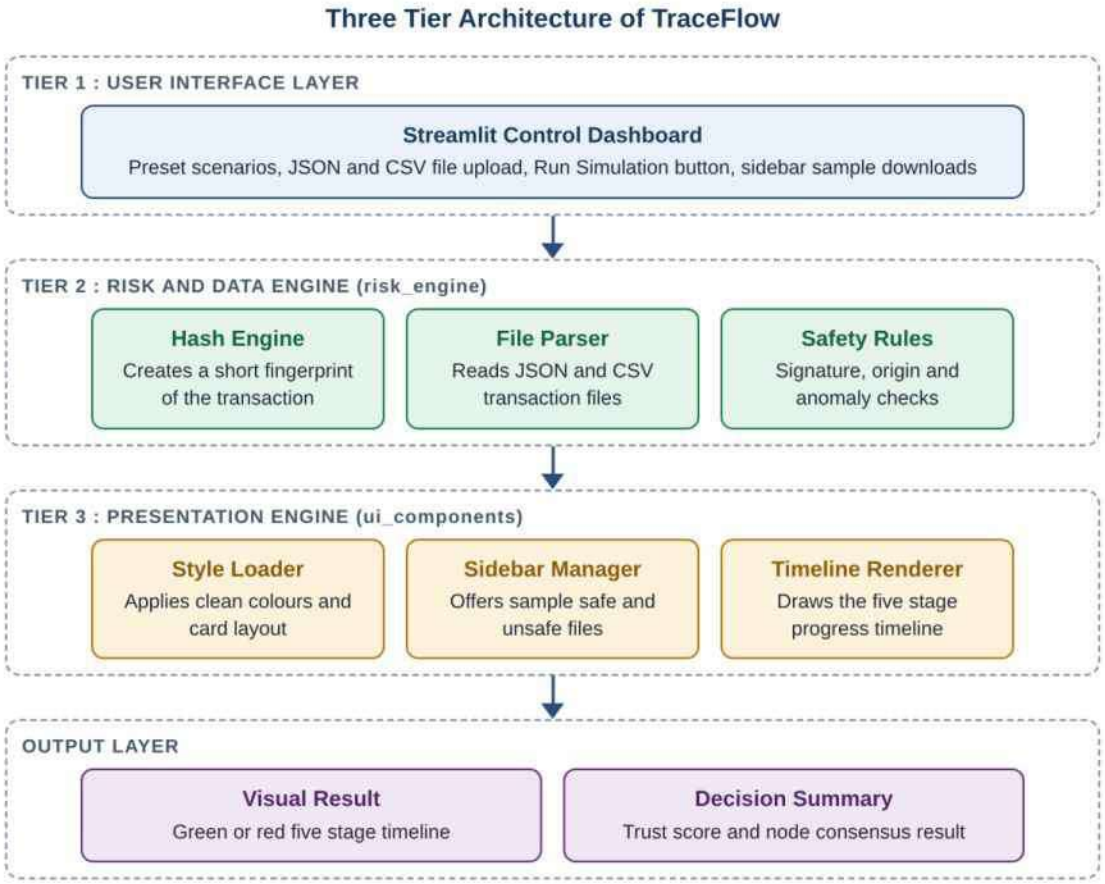
CHAPTER 3
METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 OVERALL SYSTEM ARCHITECTURE

TraceFlow is built around a three tier architecture. The principle of a tiered design is that each layer performs one category of function and communicates only with the adjacent layer. This keeps the program organised, allows each part to be tested independently and allows one part to be improved without affecting the others.

The first tier is the user interface. It is the web page presented to the user. It offers ready made preset scenarios, a file upload box for JSON and CSV files, a button to run the simulation and a sidebar where sample files can be downloaded. The second tier is the risk and data engine. It reads the file, creates the hash fingerprint and decides whether the transaction is safe. The third tier is the presentation engine. It loads the visual style, prepares the sidebar and draws the five stage progress timeline. Finally, the output layer brings together the coloured timeline, the trust score and the consensus result.

Figure 3.1 — Three Tier Architecture Diagram of TraceFlow



Information flows through the tiers in a single direction. The dashboard collects the selection made by the user, the risk engine converts it into a verdict, the presentation engine converts the verdict into a visual representation, and the output layer displays this representation together with the numerical results.

3.2 THREE TIER MODULAR SOFTWARE ARCHITECTURE

The software itself is split into three Python files. Each file has a clearly defined role.

The Risk and Data Engine (risk_engine). This is the decision-making component of the platform. It creates a twelve character hash fingerprint from the transaction text, reads the JSON or CSV file that the user supplies, pulls out the fields that matter such as origin, destination, asset, signature status, origin verification and anomaly flag, and then applies the safety rules. It does not render any output on the screen.

The UI Component Library (ui_components). This is the presentation component of the platform. It loads the style sheet that defines the appearance of the page, builds the sidebar

with download buttons for the sample files, and produces the markup for the five stage timeline in green or red. It does not make any security decision.

The Application Orchestrator (app). This is the controlling component of the application. It initialises the page, maintains the session state, shows the controls, calls the risk engine for a verdict, hands the verdict to the presentation library and runs the step by step animation with a pause between stages.

Figure 3.2 — Interaction between the Three Program Modules

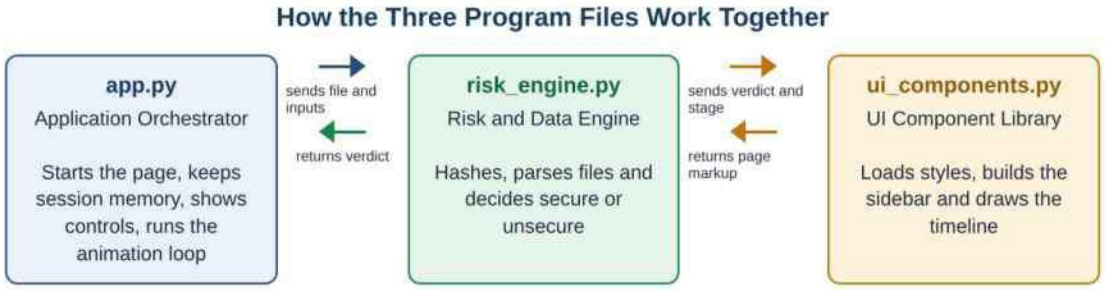


Table 3.1 — Responsibilities of the Three Program Modules

Module	Primary responsibilities	Excluded responsibilities
risk_engine	Hashing, file reading, field extraction, safety evaluation, sample file supply	No rendering or styling
ui_components	Style loading, sidebar construction, timeline rendering	No security decisions
app	Page layout, user selections, session state, invocation of the other two modules, animation loop	No safety rules or rendering code

This separation provides practical benefits. A change to the colours or wording of the timeline affects only the presentation library. A change to a rule affects only the risk engine. A new input format, such as an XML file, would need only a small addition to the file reader. The orchestrator stays almost the same in all of these cases.

3.3 SECURITY VERIFICATION RULES

The safety of a transaction is decided by three rules that are checked together. The transaction is classed as secure only when all three rules are satisfied at the same time. If even one rule fails, the transaction is classed as unsecure. In other words, the verdict is secure only when the signature is valid, the origin vendor is trusted and the anomaly check is clear.

3.3.1 Rule one: digital signature verification

A digital signature is the cryptographic proof that a transaction really came from the stated sender and has not been altered on the way. The rule passes when the signature status in the file is the word VALID, or when the digital signature text begins with the prefix 0x, which is the usual starting mark of a cryptographic signature value. A missing signature, or a signature status marked INVALID without a proper signature value, makes this rule fail.

3.3.2 Rule two: origin vendor registry check

This rule asks whether the sender is a known and registered member of the network. It passes only when the origin verified setting is true and the name of the vendor does not contain untrusted words such as Unknown or Unregistered. A sender called Unknown Vendor Node 99, for example, fails the rule immediately, even if the other fields look correct.

3.3.3 Rule three: artificial intelligence anomaly detection

This rule reads the anomaly flag that the artificial intelligence layer attaches to a transaction. It passes when the flag is NONE or empty. Any other value, such as a route anomaly, a spoofed identity or a changed batch hash, makes the rule fail. In this way the rule captures route tampering, identity spoofing and alteration of batch fingerprints.

Figure 3.3 — Decision Logic of the Three Safety Rules

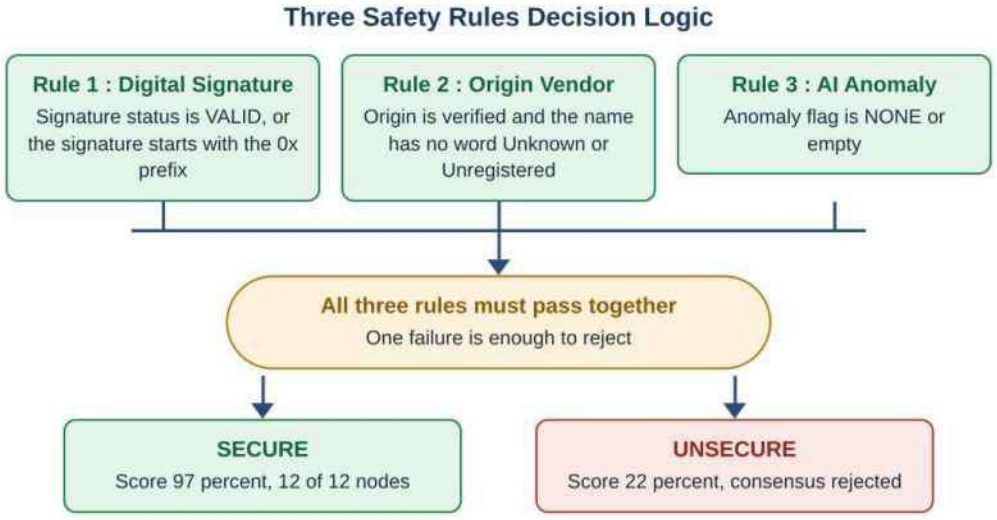


Table 3.2 — Multi Criteria Security Rule Matrix

Rule	Field examined	Passing condition	Typical failure
1 Digital signature	Signature status and digital signature text	Status is VALID, or text starts with 0x	Signature is MISSING, status INVALID

Rule	Field examined	Passing condition	Typical failure
2 Origin vendor	Origin verified setting and vendor name	Verified is true and name has no Unknown or Unregistered	Unknown Vendor Node 99
3 AI anomaly	Anomaly flag	Flag is NONE or empty	ROUTE ANOMALY SPOOFED ID

When all three rules pass, the platform assigns an AI Trust Score of ninety seven percent and shows a consensus of twelve out of twelve nodes. When any rule fails, it assigns a trust score of twenty two percent and shows that consensus was rejected, with no node agreeing.

3.3.4 Every combination of rule results

Because there are three rules and each can either pass or fail, there are eight possible combinations. The strict design gives a very clear answer for each of them, shown in the table below. Only the first row leads to acceptance.

Table 3.3 — All Eight Combinations of Rule Results and the Resulting Verdict

Signature	Origin vendor	AI anomaly	Verdict	Trust score
Pass	Pass	Pass	SECURE	97 percent
Pass	Pass	Fail	UNSECURE	22 percent
Pass	Fail	Pass	UNSECURE	22 percent
Pass	Fail	Fail	UNSECURE	22 percent
Fail	Pass	Pass	UNSECURE	22 percent
Fail	Pass	Fail	UNSECURE	22 percent
Fail	Fail	Pass	UNSECURE	22 percent
Fail	Fail	Fail	UNSECURE	22 percent

This table follows directly from the strict rule and shows that the platform has no hidden middle ground. A transaction is either trusted fully or not trusted at all. For a supply chain in which one faulty component can have serious consequences, such a strict approach is appropriate for a first line of defence.

3.4 VISUAL NETWORK PROGRESS TIMELINE DESIGN

One of the main aims of TraceFlow is that the result should be interpretable by a person without knowledge of cryptography. To achieve this, the technical life cycle of a transaction is translated into five intuitive stages and shown as a timeline of connected circles.

Secure flow. Identity Verified, then AI Analysis, then Block Minted, then In Transit, then Delivered. Every stage is shown in green.

Unsecure flow. Unverified Sender, then AI Warning, then Local Record, then In Transit, then Unverified Claim. Every stage is shown in red.

Table 3.4 — Network Stage Definitions for Secure and Unsecure Flows

Stage	Secure flow meaning	Unsecure flow meaning
1	Identity Verified: the sender is in the trusted registry	Unverified Sender: the sender cannot be confirmed
2	AI Analysis: no abnormal pattern was found	AI Warning: abnormal route or identity was flagged
3	Block Minted: all twelve nodes agree and a block is written	Local Record: consensus refused, kept only in a quarantine log
4	In Transit: the goods travel with a ledger proof	In Transit: the goods travel without a ledger proof
5	Delivered: receipt is confirmed on the ledger	Unverified Claim: the delivery claim cannot be trusted

Figure 3.4 — Five Stage Timeline Design for a Secure Flow



Figure 3.5 — Five Stage Timeline Design for an Unsecure Flow



A technical implementation detail should be noted. The timeline is produced as a block of page markup written by the presentation library. Each line of that markup is written without

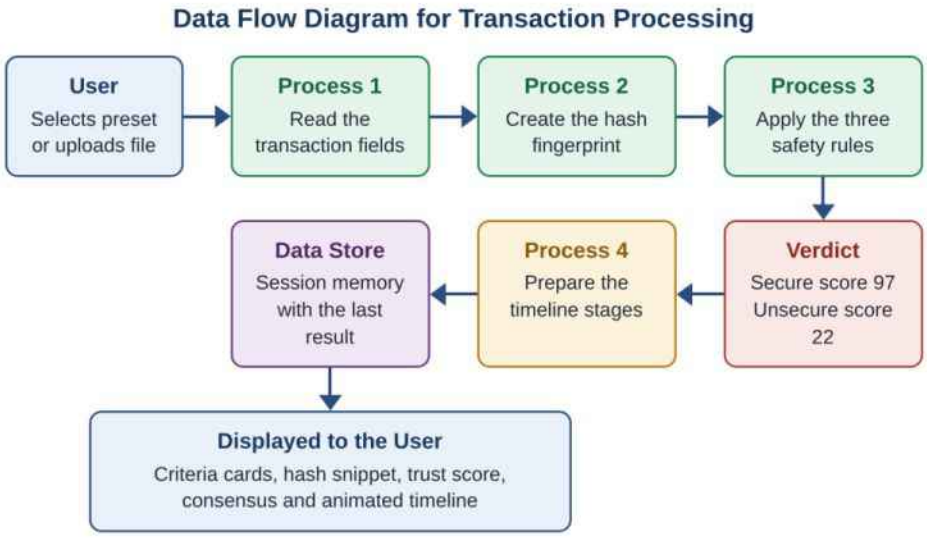
any leading blank spaces. This matters because the Streamlit framework treats an indented block as plain code to be shown as text. By removing the leading spaces, the markup is drawn as the intended coloured circles rather than printed as raw text.

The animation proceeds stage by stage. After the user presses the Run Simulation button, the orchestrator draws the timeline with only the first stage lit, waits for about one point one seconds, draws it again with two stages lit, and so on until all five are lit. The deliberate pace allows the viewer to follow the progression of the transaction and to identify precisely where a failure begins.

3.5 DATA FLOW AND PROCESS WORKFLOW

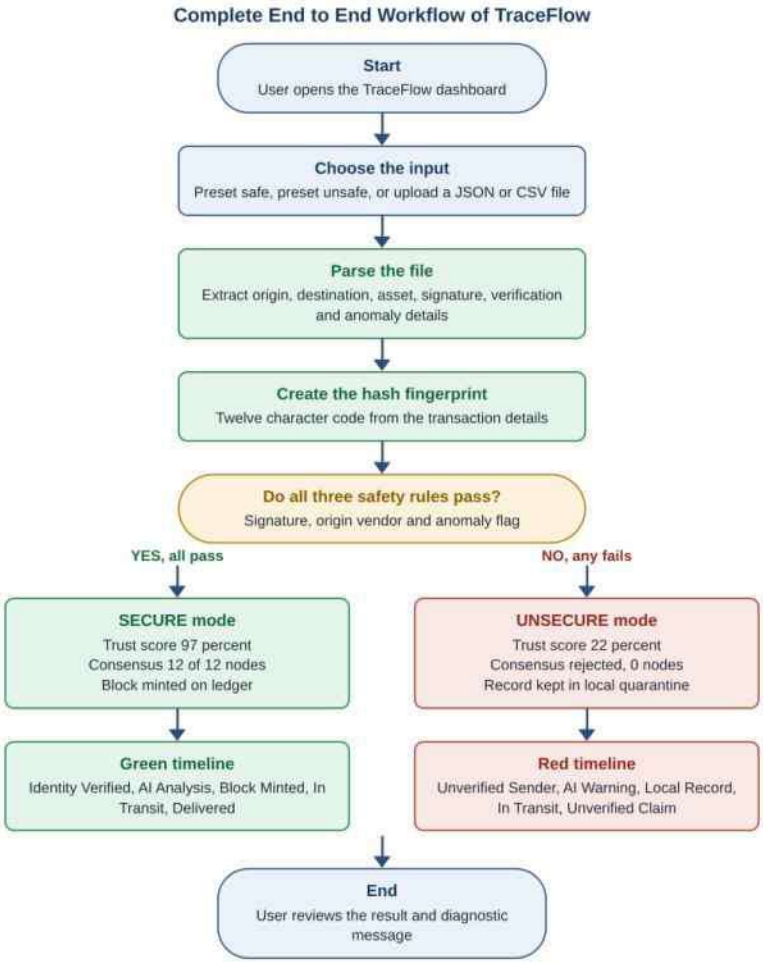
The data flow diagram below follows a transaction from the moment the user makes a choice to the moment the result is displayed. The user supplies a preset or a file. The first process reads the file and extracts the fields. The second process creates the hash fingerprint. The third process applies the three safety rules and produces the verdict, with a score of ninety seven for secure and twenty two for unsecure. The fourth process prepares the stages of the timeline. The latest result is kept in session memory so that the page can show it again if the user interacts with the page, and finally everything is displayed to the user.

Figure 3.6 — Data Flow Diagram for Transaction Processing



The complete workflow is shown in the flowchart below. It brings together the decision point, the two possible modes and the two possible timelines.

Figure 3.7 — Complete End to End Workflow of TraceFlow



3.6 HASH FINGERPRINT OF A TRANSACTION

Each transaction receives a short fingerprint. The platform joins together the key details of the transaction into one piece of text and passes that text through the Secure Hash Algorithm with a length of two hundred and fifty six bits. It then keeps the first twelve characters of the result as a compact snippet that can be shown on the screen.

The snippet serves two purposes. First, it gives the user a concrete value to observe and compare. If any detail of the transaction changes, even a single letter of the batch number, the snippet changes completely, which demonstrates visibly why tampering cannot be hidden. Second, it provides a basis for the later integration of a real ledger, where the full hash would be used to link blocks together.

3.7 INPUT HANDLING: PRESETS AND FILE UPLOAD

The platform accepts input in two ways. In preset mode the user chooses either the safe or the unsafe example from a list, and the details are filled in automatically. This mode is meant for demonstration and teaching, because a viewer can run both examples in a few seconds and compare them.

In upload mode the user supplies a JSON or CSV file. For a JSON file the engine reads the named fields directly. For a CSV file the engine reads the first data row and picks the same named columns using the Pandas library. If a field is missing, a sensible default is used, so a small mistake in the file does not cause the page to fail. The default for signature status is VALID, for origin verification is true and for anomaly flag is NONE, and the defaults for origin, destination and asset come from the values on the screen.

3.8 DESIGN DECISIONS AND RATIONALE

- **Python and Streamlit:** chosen because a full interactive web page can be built in a small number of lines, and because Python is already familiar to most engineering students.
- **Strict all-rules-pass policy:** chosen because it is easy to explain, easy to audit and appropriate for safety critical parts.
- **Separate modules:** chosen so that calculation, drawing and control can be changed and tested independently.
- **Short hash snippet:** chosen so that the result is readable on screen without long strings of characters.
- **Slow step by step animation:** chosen so that the viewer can follow the sequence rather than seeing only a final answer.
- **Colour coding:** green for safe and red for unsafe, because these colours are widely understood in industrial practice without explanation.

3.9 CONSENSUS SIMULATION AND TRUST SCORE DESIGN

In a real distributed ledger, a new block is accepted when a required number of nodes validate it. TraceFlow reproduces this idea in a simple way. The platform represents the network as twelve nodes. When the three safety rules all pass, every node is shown to agree,

and the consensus figure reads twelve out of twelve. When any rule fails, no node is shown to agree, and the consensus is reported as rejected.

The AI Trust Score summarises the same decision as a single number that is easy to compare. A passing transaction receives ninety seven percent, which is high but deliberately not one hundred percent, because no automatic check can claim perfect certainty. A failing transaction receives twenty two percent, which is low but not zero, to show that the platform does not claim to know everything about a rejected request either. In the present version these two scores are fixed values linked to the verdict. A natural improvement, described in Chapter 5, is to make the score depend on how many rules failed and on the strength of the anomaly signal.

3.10 RELIABILITY AND ERROR HANDLING

A demonstration tool must remain operational even when a user makes an error. TraceFlow therefore applies a few basic safeguards.

- **Missing fields:** if an uploaded file leaves out a field, a default value is used, so the page does not stop with an error message.
- **Letter case:** status words such as valid and VALID are converted to one standard form before they are compared.
- **Unknown file types:** only JSON and CSV files are read. Any other type is ignored rather than causing a failure.
- **Session memory:** the latest result is stored, so that interacting with the page does not erase the displayed result.

These measures do not change the safety rules. They only ensure that the platform behaves gracefully.

3.11 SUMMARY OF THE DESIGN

This chapter has set out the architecture and method of TraceFlow. The three tier layout separates thinking, drawing and control. The three rules give a strict and explainable verdict. The five stage timeline converts technical events into a sequence that any user can follow. The data flow and workflow diagrams show how a transaction moves from choice to result. The next chapter describes how this design was turned into working software and how it performed during testing.

CHAPTER 4
IMPLEMENTATION, TESTING AND RESULTS

4.1 SOFTWARE IMPLEMENTATION DETAILS

TraceFlow is written in Python version 3.13. The web page is produced with the Streamlit framework, and the Pandas library is used to read tabular CSV files. The standard hashing and JSON-handling libraries included with Python are used for the fingerprint and for reading JSON files. No database server, no container software and no special hardware are needed.

Table 4.1 — Software Environment Used for the Project

Item	Detail
Programming language	Python 3.13
Web framework	Streamlit
Tabular data reading	Pandas
Hash method	Secure Hash Algorithm, 256 bit, first twelve characters shown
Input formats	JSON and CSV files, plus preset scenarios
Number of program modules	Three: risk engine, UI components, application orchestrator
Approximate memory use	About 32 megabytes

The implementation follows the modular design strictly. Each of the three program files holds a small number of clearly named functions.

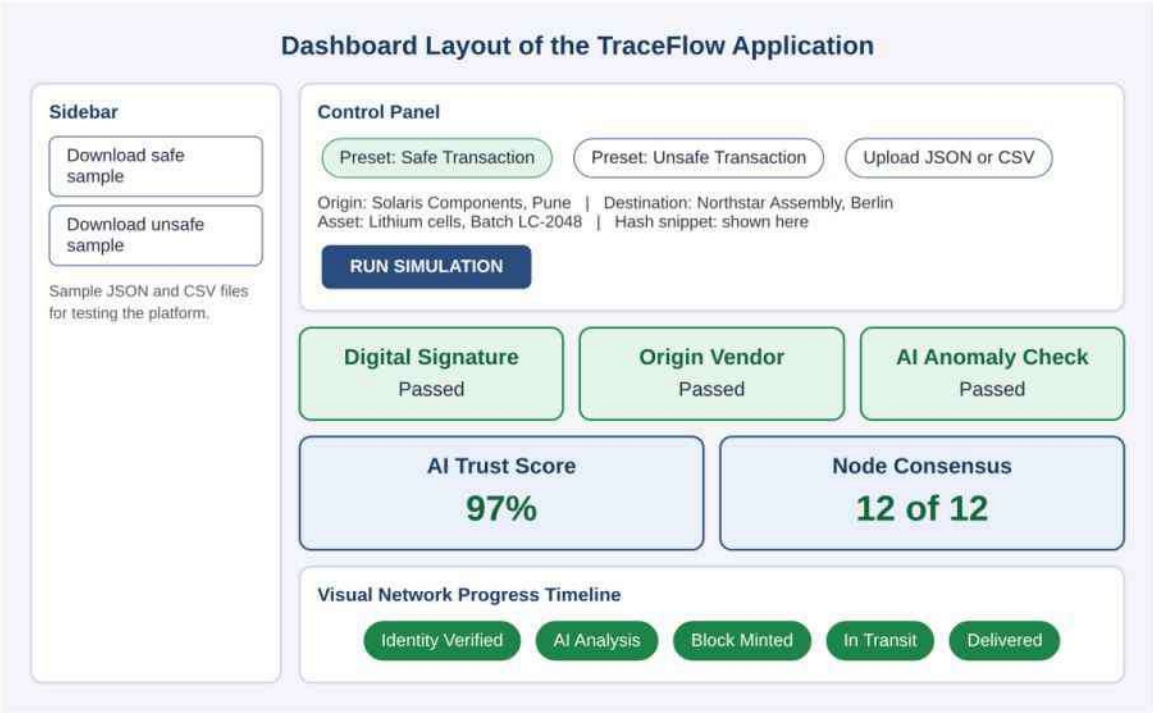
- **Risk engine functions:** one function creates the hash snippet, one reads an uploaded file and extracts the fields, one evaluates safety and returns the verdict with the result of each rule, and one supplies the sample safe and unsafe files.
- **UI component functions:** one loads the style sheet, one builds the sidebar with download buttons, and one draws the network progress timeline for a given stage and mode.
- **Application orchestrator:** arranges the page in columns, lets the user choose the mode, shows the three criteria cards, shows the trust score and consensus, and runs the animation loop.

4.2 COMPONENT INTERACTION WALKTHROUGH

The following description follows the program from the moment it starts, in the order of execution.

- **Start up:** when the application launches, the orchestrator asks the presentation library to load the visual style so that cards, circles and colours are displayed correctly from the first screen.
- **Sidebar:** the presentation library fills the sidebar with download buttons for the sample safe transaction and the sample unsafe transaction, so that a first-time user can obtain test files immediately.
- **Choice of input:** the user either picks a preset or uploads a file. If a file is uploaded, the risk engine reads it and returns the extracted fields.
- **Evaluation:** the extracted fields are passed to the safety function, which returns the final verdict and the result of each of the three rules.
- **Display of results:** the page shows the hash snippet, the three criteria cards marked Passed or Failed, the AI Trust Score and the consensus result.
- **Simulation:** when the user presses Run Simulation, the orchestrator asks the presentation library to draw the timeline for stage one, pauses, renders stage two, and continues until stage five.

Figure 4.1 — Dashboard Layout with Sidebar, Control Panel, Criteria Cards and Visualizer



The figure above illustrates how the page is arranged. The narrow sidebar on the left holds the downloads. The wide area on the right is divided into the control panel at the top, the three criteria cards, the two summary figures for trust score and consensus, and finally the timeline at the bottom. The order of these blocks follows the natural reading sequence: first what is being checked, then whether it passed, then its significance, and lastly the disposition of the goods.

4.3 EXPERIMENTAL TEST SETUP AND DATASETS

The platform was evaluated with two standard test datasets, each prepared in both JSON and CSV form. One dataset describes a correct and compliant transfer. The other describes a compromised transfer that has been deliberately altered in several respects.

Safe dataset. The origin is Solaris Components in Pune and the destination is Northstar Assembly in Berlin. The asset is a batch of lithium cells with batch number LC-2048. The digital signature is 0x8f9a2b7c4d3e1f50 and the signature status is VALID. The origin is verified and the anomaly flag is NONE.

Unsafe dataset. The origin is Unknown Vendor Node 99 and the destination is again Northstar Assembly in Berlin. The asset is the same lithium cell batch, but it is marked as

Unverified. The digital signature is MISSING and the signature status is INVALID. The origin is not verified and the anomaly flag is ROUTE ANOMALY SPOOFED ID.

Table 4.2 — Comparison of System Inputs for the Safe and Unsafe Test Runs

Field	Safe dataset	Unsafe dataset
Origin	Solaris Components, Pune	Unknown Vendor, Node 99
Destination	Northstar Assembly, Berlin	Northstar Assembly, Berlin
Asset	Lithium cells, Batch LC-2048	Lithium cells, Batch LC-2048 (Unverified)
Digital signature	0x8f9a2b7c4d3e1f50	MISSING
Signature status	VALID	INVALID
Origin verified	True	False
Anomaly flag	NONE	ROUTE ANOMALY SPOOFED ID

The unsafe dataset was designed so that all three rules fail together. This makes the contrast with the safe dataset as clear as possible. It also shows that the rules are independent: the missing signature, the unknown vendor and the spoofed route flag are each sufficient individually to cause rejection, and here all three appear at once.

4.4 SCENARIO A: COMPLIANT SAFE TRANSACTION

When the safe dataset was loaded and the simulation was run, all three criteria cards showed Passed. The signature card passed because the status was VALID and the signature began with the prefix 0x. The origin card passed because the vendor was verified and the name contained no untrusted word. The anomaly card passed because the flag was NONE.

As a result the platform assigned an AI Trust Score of ninety seven percent, showed a consensus of twelve out of twelve nodes and minted a block on the ledger. The timeline advanced through all five stages in green, from Identity Verified to Delivered, with each stage lighting up in turn.

Figure 4.2 — Visual Network Progress Timeline for SECURE Execution



4.5 SCENARIO B: COMPROMISED UNSAFE TRANSACTION

When the unsafe dataset was loaded, the criteria cards showed failures for signature validity, origin trust and anomaly check. The missing signature caused the first rule to fail. The unknown and unverified vendor caused the second rule to fail. The spoofed route flag caused the third rule to fail.

The AI Trust Score fell to twenty two percent and consensus was rejected, with zero nodes agreeing. The transaction was placed in a quarantined local log instead of the ledger. The timeline moved through the five unsecure stages in red, from Unverified Sender to Unverified Claim, accompanied by clear warning messages explaining the cause of the failure.

Figure 4.3 — Visual Network Progress Timeline for UNSECURE Execution



Figure 4.4 — AI Trust Score for the Safe and Unsafe Scenarios



Table 4.3 — Evaluation Results and Diagnostic Metrics Comparison

Measure	Scenario A: Safe	Scenario B: Unsafe
Signature rule	Passed	Failed
Origin vendor rule	Passed	Failed
AI anomaly rule	Passed	Failed
Overall verdict	SECURE	UNSECURE
AI Trust Score	97 percent	22 percent
Node consensus	12 of 12 nodes	Rejected, 0 nodes
Ledger action	Block minted	Quarantined in local log
Timeline colour	Green	Red
Classification correct	Yes	Yes

Both scenarios were classified correctly in every one of the test runs, which gives a classification accuracy of one hundred percent on the datasets used. It should be understood that this accuracy refers to the prepared safe and unsafe examples. It confirms that the rules are implemented correctly and behave as designed. Broader claims about accuracy on real world data would require a larger collection of real transactions.

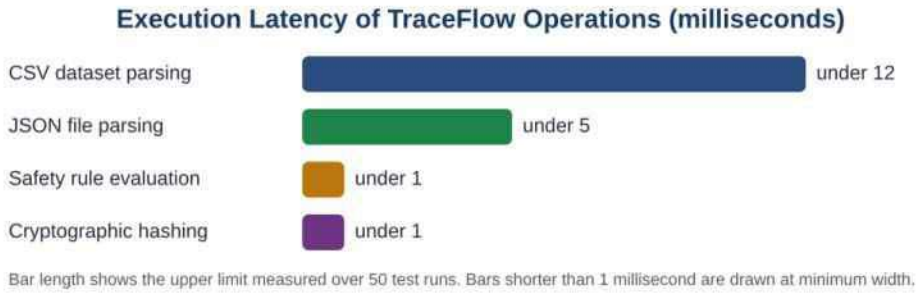
4.6 PERFORMANCE AND EXECUTION LATENCY

The speed of the platform was measured over fifty test runs. The platform responded rapidly in every case, as shown in the table below.

Table 4.4 — Execution Latency and Memory Utilization

Operation	Measured result
JSON file parsing time	Less than 5 milliseconds
CSV dataset parsing time	Less than 12 milliseconds
Cryptographic hashing latency	Less than 1 millisecond
Total memory footprint	About 32 megabytes of RAM

Figure 4.5 — Execution Latency of the TraceFlow Operations



Reading a JSON file is the fastest operation because the structure is already organised into named fields. A CSV file takes slightly longer because the Pandas library must construct a table before the first row can be extracted. Hashing is nearly instantaneous. Taken together, the whole verification step is completed in much less than a fiftieth of a second. The gradual progression of the timeline, with a pause of about one second per stage, is a deliberate design choice intended for the viewer and is unrelated to computational delay.

The small memory footprint is also important. At about thirty two megabytes the platform can run on a modest laptop or on a small edge computer, which supports the aim of keeping the platform light and easy to deploy.

4.7 DISCUSSION OF RESULTS

The results support the main claims of the project. The platform reached the correct decision in both scenarios, it gave the reason for each decision through the three criteria cards, it responded within a few milliseconds, and it used very little memory.

A particular strength is explainability. In the unsafe scenario the user is not told only that the transaction failed. The user is told which of the three rules failed and sees the failure reflected in the red timeline. This helps an auditor trace the cause of a rejection, and it helps a trainee learn what each rule protects against.

The strictness of the design is both a strength and a limit. It is a strength because no partly suspicious transaction can pass undetected. It is a limit because an inadvertent data-entry error, such as a missing optional field being read as an invalid signature, could cause a good transaction to be rejected. In a production setting a manual review path for rejected transactions would be useful, and this is mentioned among the suggestions in Chapter 5.

4.8 ILLUSTRATIVE USE CASE

To illustrate how the platform would be used in a real setting, consider a battery pack factory in Berlin that expects a batch of lithium cells from its approved supplier in Pune. When the shipment is dispatched, the supplier prepares a transaction file describing the batch, signs it digitally and uploads it to the platform. The platform reads the file, creates the fingerprint, and finds that the signature is valid, the supplier is in the registry and no anomaly is flagged. The twelve nodes agree, a block is created and the goods start their journey with a ledger proof. When the goods arrive, the receiving team sees a green delivered stage and can proceed to inspection with confidence.

Consider now a case in which a company that is not in the registry attempts to send a batch of cells under a similar name. Its file has no signature, its name is not recognised and the route differs from any known supply route. The platform fails all three rules, shows a red timeline and a trust score of twenty two percent, and keeps the request in a quarantine log. The purchasing team receives a clear warning before any goods are accepted into the production line, and the security team has a record to investigate.

The two cases show the value of the design. In the first, the honest supplier loses no time, because the check takes only a few milliseconds. In the second, the dishonest request is stopped at the first checkpoint, before it can cause any harm.

4.9 LIMITATIONS OF THE PRESENT WORK

- **Simulated ledger:** the network consensus and block minting are simulated inside the application and are not recorded on a real distributed ledger.
- **Rule based intelligence:** the artificial intelligence check works by reading an anomaly flag that is supplied with the transaction. It does not yet learn from data autonomously.
- **Small test set:** the evaluation used the standard safe and unsafe datasets, not a large collection of real transactions.

- **Single transaction at a time:** the platform examines one transaction per run and does not yet process a continuous stream.

4.10 SUMMARY

This chapter has described the software environment, the walkthrough of the components, the test datasets, the outcome of the two scenarios and the measured speed. TraceFlow classified both test transactions correctly, gave clear reasons, and ran in a few milliseconds with a small memory footprint. The limitations that were noted lead naturally to the future work described in the final chapter.

CHAPTER 5
CONCLUSION AND SCOPE FOR FUTURE WORK

5.1 SUMMARY OF RESEARCH ACCOMPLISHMENTS

The TraceFlow platform shows that an artificial intelligence guarded supply chain ledger can be built in a simple, light and understandable form. The main accomplishments of the project are as follows.

- A working three tier Python application was developed, made up of a risk engine, a UI component library and an application orchestrator, each with its own clear job.
- A strict three rule safety framework was established, combining digital signature checking, origin vendor registry checking and artificial intelligence anomaly checking into a single clear verdict.
- An intuitive visual interface was created in which complex ledger events are presented as a five stage coloured timeline, simple pass or fail cards, a trust score and a consensus figure.
- The platform was tested with safe and unsafe datasets in both JSON and CSV form and classified both correctly, giving one hundred percent accuracy on these datasets.
- The platform was shown to be fast and lightweight, with file reading in under twelve milliseconds and memory use of about thirty two megabytes.

5.2 KEY TAKEAWAYS AND PRACTICAL CONTRIBUTIONS

The central lesson of the project is that a lightweight, inline rule check combined with a clear visual display can serve as a practical alternative to resource-intensive blockchain installations when the aim is learning, early prototyping and rapid monitoring. The platform shows that the check must come before the record, that the reasons for a decision must be visible, and that the display must be designed for human users rather than for machines.

The practical contributions can be grouped under three headings.

For industry. Supply chain teams can use the platform to explore how signature, vendor and anomaly checks would work in their own material handover points before investing in a full production network.

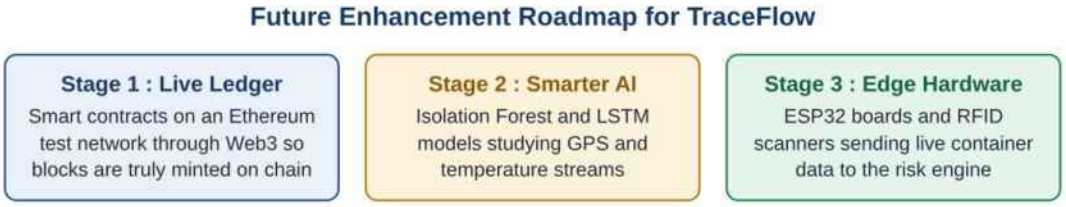
For education. Teachers can use the two sample datasets and the animated timeline to demonstrate how a ledger network accepts or refuses a transfer. The ability to download and edit the files encourages students to experiment with their own changes.

For research. The modular structure provides a convenient base on which more advanced models, real ledgers and live sensors can be added one at a time.

5.3 SCOPE FOR FUTURE WORK AND ENHANCEMENTS

Three directions of improvement are suggested, as shown in the roadmap below.

Figure 5.1 — Future Enhancement Roadmap for TraceFlow



Live smart contract integration. Smart contracts can be deployed on a public or permissioned Ethereum test network such as Sepolia and connected to the platform through the Web3 library. This would allow blocks to be truly minted on a chain rather than simulated, and would give the platform a real transaction history that can be inspected by outside parties.

Advanced artificial intelligence models. The simple anomaly flag can be replaced by trained models. An Isolation Forest could rate each transaction by how unusual it is compared with normal ones, and a Long Short Term Memory network could study continuous streams of location and temperature readings to detect a journey that has gone wrong, for example a container whose temperature has exceeded the safe range for lithium cells.

Edge hardware integration. ESP32 microcontrollers and RFID scanners can be connected so that real container sensor readings flow straight into the risk engine. This would link the physical world to the digital ledger and remove the need for any manual data entry at the handover point.

Further useful additions include a manual review queue for rejected transactions, processing of many transactions in a continuous stream, a graded trust score based on how many rules failed rather than a fixed value, and user accounts with different views for managers, auditors and operators.

5.4 CONCLUDING REMARKS

Supply chains depend on trust between parties who may never meet. Technology can support that trust only when it is strong, fast and understandable. TraceFlow demonstrates that these three qualities can be brought together in a small program that runs on a standard computer. By stopping suspicious transfers before they are recorded, by showing the reason for each decision, and by presenting the progression of every transaction in a manner that any user can follow, the project represents a useful step towards safer and more transparent industrial supply chains.

PROJECT OUTCOMES

The project has produced the following outcomes.

- **Working software system:** a complete and fully functional application built with Python and Streamlit, made up of three modules.
- **Standardized test datasets:** downloadable safe and unsafe transaction sample files in both JSON and CSV form.
- **Visual explanation tools:** a five stage animated timeline for secure and unsecure flows, together with architecture, data flow and workflow diagrams.
- **Academic documentation:** a project report prepared in line with the format followed by Rajalakshmi Engineering College.

APPENDIX I : DESCRIPTION OF THE PROGRAM MODULES

This appendix describes the functions that make up each of the three program modules. The descriptions specify the input, processing and output of each function.

A1.1 Risk and Data Engine

Table A1 — Functions of the Risk and Data Engine

Function	Input	Description	Output
Hash generator	A piece of transaction text	Applies the 256 bit secure hash method and keeps the first twelve characters	A short fingerprint
File reader	An uploaded JSON or CSV file and default values	Reads the named fields, uses default values for any missing field, and converts text to a standard form	Origin, destination, asset, signature status, origin verified, anomaly flag, digital signature
Safety evaluator	Signature status, signature text, origin verified, origin name, anomaly flag	Applies the three rules and combines them so that all must pass	Overall verdict and the result of each rule
Sample provider	None	Prepares the safe and unsafe sample files for download	Sample files in JSON and CSV form

A1.2 UI Component Library

Table A2 — Functions of the UI Component Library

Function	Description
Style loader	Applies the colours, card shapes and circle styles to the page to ensure a consistent appearance
Sidebar builder	Creates the sidebar with buttons to download the sample safe and unsafe files
Timeline renderer	Renders the five-stage timeline for the chosen mode, lighting up stages up to the current step in green or red

A1.3 Application Orchestrator

The orchestrator is the entry point of the application. It arranges the page into columns, shows the mode selection and file upload controls, calls the file reader and the safety evaluator, displays the three criteria cards, the trust score and the consensus result, and runs the loop that

draws the timeline one stage at a time with a pause between stages. It also keeps the latest result in session memory.

APPENDIX II : STANDARDIZED DATASET FIELDS

Both the JSON and the CSV test files use the same set of named fields. A file may leave out a field, in which case the default shown below is applied.

Table A3 — Fields of a Transaction Dataset

Field name	Meaning	Default when missing
origin	Name of the sending vendor and location	Value selected on the screen
destination	Name of the receiving plant and location	Value selected on the screen
asset	Description of the material and its batch	Value selected on the screen
digital_signature	The cryptographic signature text	Value selected on the screen
signature_status	VALID or INVALID	VALID
origin_verified	True if the vendor is in the registry	True
anomaly_flag	NONE, or the name of the anomaly found	NONE

The sample safe file describes the transfer of lithium cell batch LC-2048 from Solaris Components in Pune to Northstar Assembly in Berlin with a valid signature, a verified origin and no anomaly. The sample unsafe file describes the same batch from an unknown vendor with a missing signature, an unverified origin and a route anomaly flag.

REFERENCES

[1] S. E. Chang and Y. Chen, "When blockchain meets supply chain: A systematic literature review on current development and potential applications," *IEEE Access*, vol. 8, pp. 2983601, 2020.

[2] R. Vettriselvan, C. Vijai, J. D. Patel, S. Kumar R., P. Sharma, and N. Kumar, "Blockchain embraces supply chain optimization for enhancing transparency and traceability from production to delivery," in *Proc. Int. Conf. Trends Quantum Comput. Emerg. Bus. Technol. (TQCEBT)*, Pune, India, 2024, pp. 1–6.

[3] S. J. Divey, M. H. Hekimoğlu, and T. Ravichandran, "Blockchains in supply chains: Potential research directions," in *Proc. IEEE Technol. Eng. Manage. Conf. (TEMSCON)*, Atlanta, GA, USA, 2019, pp. 1–6.

[4] U. Agarwal, V. Rishiwal, M. Yadav, M. Alshammari, P. Yadav, O. Singh, and V. Maurya, "Exploring blockchain and supply chain integration: State-of-the-art, security issues, and emerging directions," *IEEE Access*, vol. 12, pp. 143945–143974, 2024.

[5] R. Zhu, W. Zheng, X. Zhang, and C. Chen, "Blockchain-based traceability system of broiler product quality and safety," in *Proc. 9th Int. Conf. Electron. Technol. Inf. Sci. (ICETIS)*, 2024, pp. 619–626.

[6] S. Gupta, H. Sharma, V. Hassija, and V. Saxena, "BitCom: A commerce model on blockchain," in *Proc. IEEE Int. Conf. Contemporary Comput. (IC3)*, Noida, India, 2020, pp. 64–70.

[7] Baranidharan A and P. Velmurugadoss, "Federated learning and blockchain for predictive healthcare," in *Proc. 7th Int. Conf. Comput., Commun. Autom. (ICCCA)*, 2025, pp. 1–5.

[8] D. Kasimatis, I. Politis, N. Pitropakis, P. Papadopoulos, and W. J. Buchanan, "Decentralized device identity: PUF-driven Soulbound token verification for IoT supply chain security," *IEEE Trans. Consum. Electron.*, vol. 72, no. 1, pp. 332–341, Feb. 2026.

[9] S. Agarwal, S. Alok, and R. Handaragal, "From prediction to fulfillment: A real-time AI planning system for modern supply chains," in *Proc. IEEE North Karnataka Subsection Flagship Int. Conf. (NKCon)*, 2025, pp. 1–6.

[10] H. Liu, P. Jiang, W. Tao, and Y. Dai, "Community education authentication system based on blockchain technology," in *Proc. 12th Joint Int. Inf. Technol. Artif. Intell. Conf. (ITAIC)*, 2025, pp. 295–299.